Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: «Современные платформы программирования»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

Веб-приложение для изучения иностранных языков

БГУИР КП 6-05-0612-01 123 ПЗ

Студент Ушаков А.Д.

Руководитель Петровская В.В.

Минск 2026

СОДЕРЖАНИЕ

Введение 7
1 Анализ предметной области и существующих аналогов 8
1.1 Обзор аналогов 8
1.2 Сравнительный анализ аналогов 12
1.3 Постановка задачи 13
2 Анализ требований к программному средству и разработка функциональных требований 14
2.1 Назначение программного средства 14
2.2 Функциональные требования 14
2.3 Нефункциональные требования 15
2.4 Требования к входным и выходным данным 15
2.5 Требования к интерфейсу 16
3 Проектирование программного средства 17
3.1 Архитектура программного средства 17
3.2 Проектирование базы данных 17
3.3 Проектирование REST API 18
3.4 Проектирование интерфейса 19
3.5 Алгоритмы функционирования 19
4 Создание (конструирование) программного средства 27
4.1 Средства разработки 27
4.2 Реализация аутентификации 28
4.3 Реализация CRUD и поиска 30
4.4 Интеграция со сторонним сервисом перевода 32
4.5 Реализация уведомлений SSE 33
4.6 Адаптивная верстка 35
5 Тестирование, проверка работоспособности и анализ полученных результатов 36
5.1 Виды тестирования 36
5.2 Тест-кейсы 36
5.3 Анализ результатов 37

6 Руководство по установке и использованию 38
6.1 Установка и запуск 38
6.2 Руководство пользователя..... 39
Заключение 42
Список использованных источников 43
ПРИЛОЖЕНИЕ А (обязательное) Листинги программного средства.. 44

ВВЕДЕНИЕ

Изучение иностранных языков является важным условием личного и профессионального развития, а цифровые технологии делают этот процесс более доступным и удобным. Современные веб-приложения позволяют пользователю заниматься в удобное время, работать с собственными учебными материалами и отслеживать прогресс. В связи с этим разработка веб-приложения для изучения иностранных языков является актуальной задачей.

Целью курсового проекта является создание веб-приложения для изучения иностранных языков, которое обеспечивает регистрацию и авторизацию пользователей, работу с учебными материалами, поиск и сортировку записей, отслеживание прогресса, интеграцию со сторонним сервисом перевода и отправку уведомлений. Для достижения цели необходимо проанализировать предметную область и существующие аналоги, сформулировать требования к программному средству, спроектировать архитектуру и базу данных, реализовать клиентскую и серверную части, провести тестирование и подготовить руководство по установке и использованию.

При разработке используются Next.js с TypeScript, REST API на Route Handlers, MongoDB с Mongoose, шифрование паролей с помощью bcrypt, механизм Server-Sent Events, сторонний сервис перевода MyMemory, а также Git и GitHub. Интерфейс приложения должен быть русскоязычным, адаптивным и удобным как на персональных компьютерах, так и на мобильных устройствах [1, 2].

Практическая значимость проекта заключается в создании инструмента, который позволяет систематизировать изучение иностранного языка, хранить слова и уроки, отслеживать прогресс и получать своевременные уведомления. Пояснительная записка включает введение, шесть разделов, заключение, список использованных источников и приложения. В первом разделе выполнен анализ предметной области и аналогов. Во втором разделе сформулированы требования к программному средству. В третьем разделе приведено проектирование архитектуры, данных, интерфейса и алгоритмов. В четвёртом разделе описано конструирование программного средства. В пятом разделе представлены результаты тестирования. В шестом разделе приведено руководство по установке и использованию.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ АНАЛОГОВ

1.1 Обзор аналогов

Изучение иностранного языка включает запоминание слов, освоение грамматики, развитие навыков чтения, аудирования, письма и речи. Веб-приложения позволяют организовать этот процесс гибко и индивидуально. Для определения особенностей собственной разработки необходимо рассмотреть существующие аналоги, выявить их сильные и слабые стороны, а затем сформулировать требования к проектируемому программному средству.

1.1.1 Программное средство Duolingo. Среди наиболее известных решений для изучения иностранных языков можно выделить Duolingo, Lingualeo, Quizlet и Anki. Они различаются по функциональности, способам подачи материала, наличию геймификации и возможностям работы с пользовательским контентом.

Duolingo представляет собой популярную платформу с короткими уроками, упражнениями на перевод, аудирование и выбор правильного ответа. Система использует баллы, уровни, достижения и серии ежедневных занятий, что повышает мотивацию. К преимуществам относятся бесплатный базовый доступ, понятный интерфейс и большое количество языков. Однако платформа ориентирована на стандартизированные курсы, слабо поддерживает создание собственных учебных материалов и ограниченно представляет сложную грамматику и профессиональную лексику. Интерфейс веб-приложения Duolingo показан на рисунке 1.1.

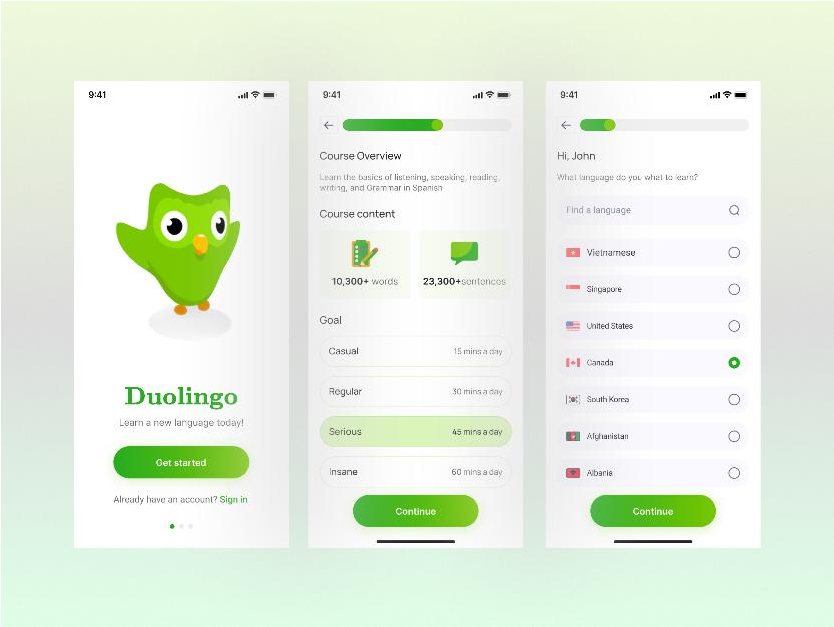


Рисунок 1.1 – Интерфейс веб-приложения Duolingo

1.1.2 Программное средство Lingualeo. Lingualeo ориентирован в том числе на русскоязычных пользователей и предлагает интерактивные упражнения, видео, тексты, грамматические курсы и персональный словарь. Пользователь может добавлять незнакомые слова и тренировать их в разных режимах. К достоинствам относятся русскоязычный интерфейс, разнообразие тренировок и работа с реальными материалами. К недостаткам можно отнести ограничения бесплатной версии, преимущественную ориентацию на английский язык и ограниченные возможности настройки индивидуального учебного плана. Интерфейс приложения Lingualeo показан на рисунке 1.2.

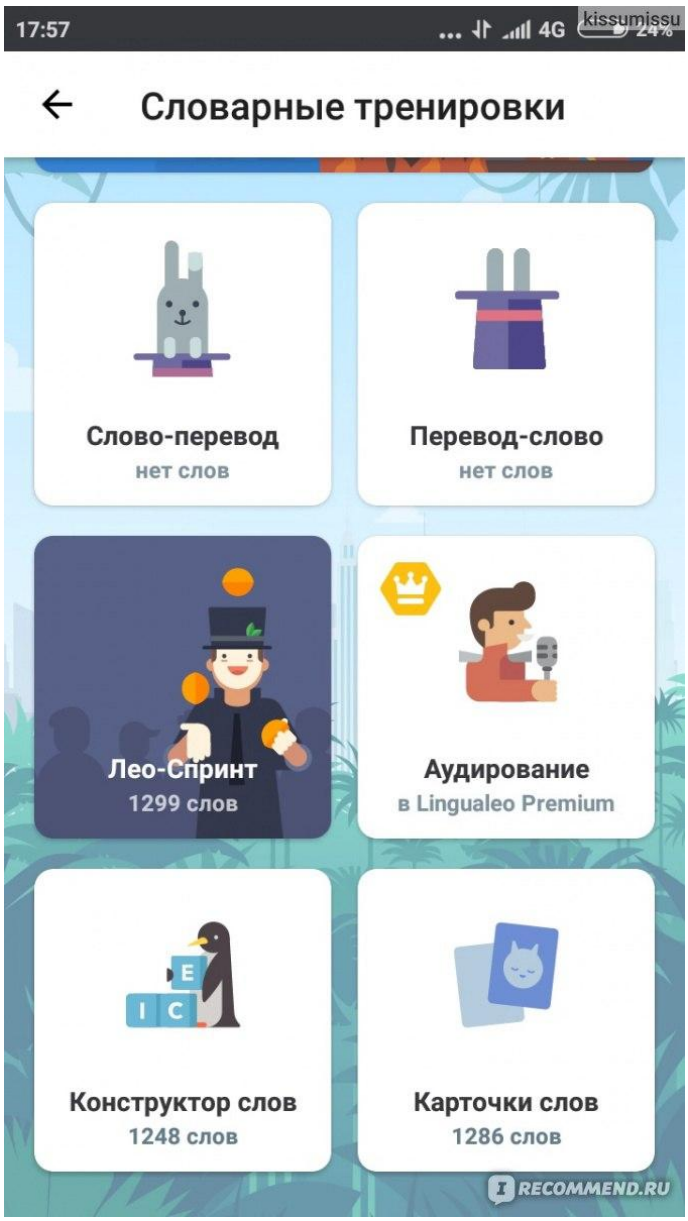


Рисунок 1.2 – Интерфейс веб-приложения Lingualeo

1.1.3 Программное средство Quizlet. Quizlet предназначен для создания и использования учебных карточек, тестов и игровых заданий. Пользователи могут формировать собственные наборы карточек, добавлять термины и определения, использовать режимы заучивания и проверки знаний. Сервис гибок и удобен для повторения, но не является полноценным курсом иностранного языка, поскольку в нём мало внимания уделяется грамматике, аудированию и разговорной практике. Интерфейс Quizlet показан на рисунке 1.3.

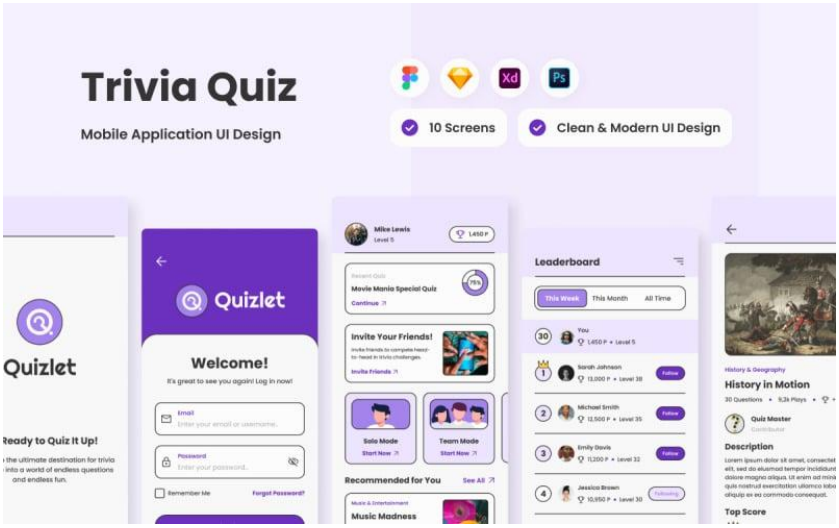


Рисунок 1.3 – Интерфейс веб-приложения Quizlet

1.1.4 Программное средство Anki. Anki использует карточки и интервальное повторение для эффективного запоминания информации. Приложение поддерживает создание собственных колод, импорт и экспорт данных, теги, статистику и синхронизацию. К преимуществам относятся высокая эффективность повторения и гибкость настройки. Однако интерфейс может быть сложным для начинающих, отсутствует готовый структурированный курс языка, а создание качественных материалов требует времени и методических знаний. Интерфейс Anki показан на рисунке 1.4.

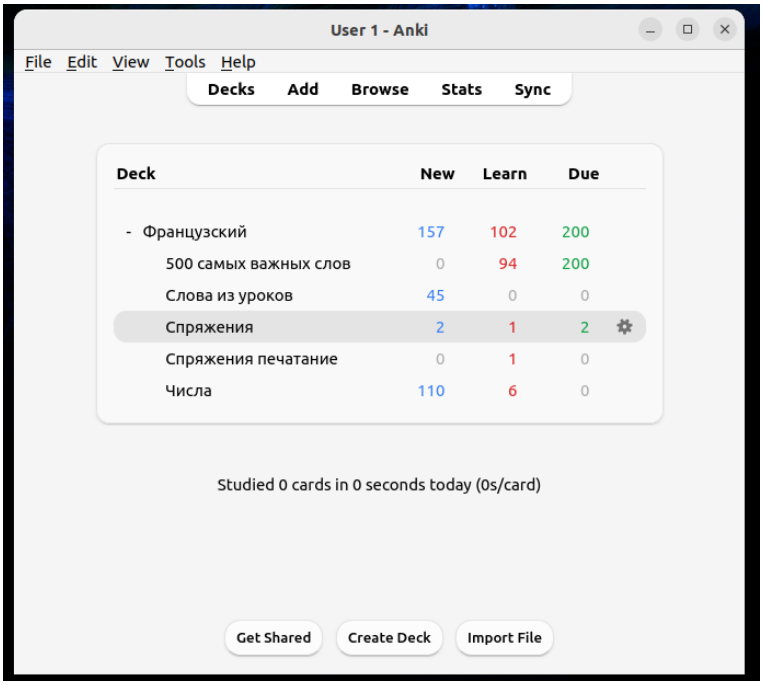


Рисунок 1.4 – Интерфейс приложения Anki

1.2 Сравнительный анализ аналогов

Рассмотренные приложения обладают развитым функционалом, но каждое имеет ограничения. Duolingo и Lingualeo предлагают структурированные уроки и элементы геймификации, однако значительная часть возможностей доступна по подписке, а работа с собственными материалами ограничена. Quizlet и Anki хорошо подходят для запоминания и повторения, но не дают полноценного курса иностранного языка и требуют самостоятельного создания контента. Кроме того, не все аналоги обеспечивают удобную интеграцию перевода, уведомления и адаптивный личный кабинет с разделами для слов, уроков и прогресса. Сравнение аналогов по ключевым признакам приведено в таблице 1.1.

Таблица 1.1 – Сравнение аналогов проектируемого программного средства

Признак	Duolingo	Lingualeo	Quizlet	Anki
Собственные материалы	ограничено	частично	да	да
Русскоязычный интерфейс	частично	да	частично	частично
Поиск и сортировка записей	ограничено	частично	да	да
Интеграция перевода	ограничено	частично	нет	нет

Признак	Duolingo	Lingualeo	Quizlet	Anki
Уведомления в реальном времени	нет	нет	нет	нет
Открытый исходный код проекта	нет	нет	нет	да

Анализ таблицы 1.1 показывает, что ни один из рассмотренных аналогов одновременно не сочетает полный CRUD учебных материалов, встроенный перевод, уведомления и развёртывание в виде собственного веб-приложения. Это подтверждает целесообразность разработки программного средства Lingua, которое объединит необходимые функции в единой системе.

1.3 Постановка задачи

На основе анализа предметной области и существующих аналогов необходимо разработать веб-приложение для изучения иностранных языков с использованием Next.js и TypeScript, REST API на Route Handlers, MongoDB с Mongoose, bcrypt для шифрования паролей и Server-Sent Events для уведомлений [2–4]. Приложение должно обеспечивать регистрацию и вход пользователей, проверку входящих данных, создание, редактирование, удаление и просмотр учебных материалов, работу со словами и уроками, поиск и сортировку записей, личный кабинет с разделами для изучения слов, уроков и прогресса, отслеживание прогресса, интеграцию со сторонним сервисом перевода, отправку уведомлений через SSE, адаптивную верстку и хранение данных в MongoDB. Проект должен вестись в Git с размещением репозитория на GitHub. В результате предполагается создать полноценное веб-приложение с русскоязычным интерфейсом, которое позволит пользователю систематизировать изучение иностранного языка, работать с собственными материалами и отслеживать свои результаты.

2 АНАЛИЗ ТРЕБОВАНИЙ К ПРОГРАММНОМУ СРЕДСТВУ И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

2.1 Назначение программного средства

Программное средство Lingua предназначено для индивидуальной работы пользователя с учебными материалами по иностранному языку. Система должна позволять регистрировать учётную запись, входить в личный кабинет, вести словарь, формировать уроки, контролировать прогресс, получать перевод отдельных слов и получать уведомления о значимых действиях. Программное средство ориентировано на эксплуатацию в браузере и разворачивается в контейнерной среде Docker.

Потребителем является обучающийся, который самостоятельно формирует словарный запас и набор уроков. Административная роль в рамках курсового проекта не выделяется: каждый пользователь работает только со своими данными.

2.2 Функциональные требования

На основании задания на курсовое проектирование и результатов анализа аналогов сформулированы функциональные требования (ФТ). Программное средство должно обеспечивать:

– ФТ-1 регистрацию пользователя по имени, почте и паролю;
– ФТ-2 вход в систему и завершение сессии;
– ФТ-3 хранение сессии на основе JWT в защищённой cookie;
– ФТ-4 создание, просмотр, изменение и удаление словарных записей;
– ФТ-5 создание, просмотр, изменение и удаление уроков;
– ФТ-6 поиск записей по тексту и сортировку по выбранному полю;
– ФТ-7 проверку входящих данных на стороне сервера;
– ФТ-8 шифрование паролей алгоритмом bcrypt;
– ФТ-9 отображение прогресса по словарю и урокам;
– ФТ-10 режим повторения слов с изменением уровня усвоения;
– ФТ-11 обращение к стороннему сервису перевода;
– ФТ-12 доставку уведомлений по протоколу SSE;
– ФТ-13 адаптивное отображение интерфейса на узких экранах;
– ФТ-14 разграничение данных по идентификатору пользователя.

1) Комментарий к ФТ-1 и ФТ-2. Регистрация требует имени длиной не менее двух символов, корректной почты и пароля длиной не менее восьми

символов. Повторная регистрация с той же почтой должна отклоняться. Вход выполняется по почте и паролю; при несовпадении возвращается ошибка без уточнения, какое именно поле неверно.

2) Комментарий к ФТ-4 и ФТ-5. Словарная запись содержит термин, перевод, необязательные транскрипцию, часть речи, пример, теги, язык и уровень усвоения от 0 до 5. Урок содержит название, текст, язык и уровень A1–C2. Операции выполняются только для записей текущего пользователя.

3) Комментарий к ФТ-11. Перевод выполняется через HTTP-запрос к сервису MyMemory. При недоступности внешнего сервиса клиент получает сообщение об ошибке, локальные данные не повреждаются [5].

4) Комментарий к ФТ-12. Клиент кабинета открывает поток `text/event-stream`. При создании, изменении и удалении записей сервер сохраняет уведомление в MongoDB и сразу отправляет событие подключённому клиенту.

2.3 Нефункциональные требования

К программному средству предъявляются следующие нефункциональные требования:

– НФТ-1 клиентская и серверная части реализуются на TypeScript в рамках Next.js;

– НФТ-2 программный интерфейс строится как REST API на Route Handlers;

– НФТ-3 данные хранятся в MongoDB и описываются схемами Mongoose [3];

– НФТ-4 интерфейс выполняется на русском языке;

– НФТ-5 запуск выполняется командой `docker compose up`;

– НФТ-6 исходный код сопровождается репозиторием Git;

– НФТ-7 время ответа локального API при типовых операциях CRUD не должно превышать нескольких секунд на учебной нагрузке.

2.4 Требования к входным и выходным данным

Входными данными являются заполняемые пользователем формы регистрации, входа, словаря, уроков и перевода, а также параметры поиска и сортировки в строке запроса. Выходными данными являются JSON-ответы REST API и HTML-страницы приложения. Форматы запросов приведены в таблице 2.1.

Таблица 2.1 – Основные входные и выходные данные

Операция	Вход	Выход
Регистрация	JSON: name, email, password	JSON: user; cookie сессии
Вход	JSON: email, password	JSON: user; cookie сессии
Список слов	q, sort, order	JSON: items[]
Создание слова	JSON полей записи	JSON: item; уведомление SSE
Перевод	JSON: text, from, to	JSON: translation
Поток уведомлений	cookie сессии	text/event-stream

Некорректные данные должны отклоняться с кодом 400, отсутствие сессии – с кодом 401, конфликт почты – с кодом 409, ошибка внешнего сервиса – с кодом 502.

2.5 Требования к интерфейсу

Интерфейс должен содержать стартовую страницу, формы входа и регистрации, личный кабинет с разделами «Обзор», «Слова», «Уроки» и «Прогресс». В шапке кабинета отображаются название приложения, имя пользователя и команда выхода. Раздел «Слова» включает форму создания и таблицу записей. Раздел «Уроки» включает форму и карточки материалов. Раздел «Прогресс» отображает количественные показатели и форму повторения. Уведомления показываются в кабинете и во всплывающих сообщениях. Цветовое оформление выполняется в спокойной учебной гамме без зависимости от внешнего UI-набора.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Архитектура программного средства

Программное средство строится как одно приложение Next.js, в котором клиентские страницы и серверные Route Handlers размещаются в общем проекте. Такой подход соответствует заданию: разработку ведут на TypeScript для клиентской и серверной частей, а API реализуют через REST на Route Handlers [2]. Внешними узлами являются СУБД MongoDB и сервис перевода MyMemory. Состав программных частей:

– слой представления – страницы App Router, формы и таблицы кабинета;

– слой прикладного API – обработчики `/api/auth`, `/api/words`, `/api/lessons`, `/api/progress`, `/api/translate`, `/api/notifications`;

– слой данных – модели Mongoose User, Word, Lesson, Notice;

– слой интеграции – HTTP-клиент перевода и диспетчер SSE.

Клиент обращается к API того же происхождения, поэтому cookie сессии передаётся браузером автоматически. Контейнер приложения и контейнер MongoDB связываются сетью Docker Compose.

В состав серверной части входят следующие модули:

– `src/lib/db.ts` – подключение к MongoDB с повторными попытками;

– `src/lib/auth.ts` – формирование и проверка JWT, работа с cookie;

– `src/lib/validate.ts` – проверка строк, почты, диапазона усвоения, белого списка сортировки;

– `src/lib/sse.ts` и `src/lib/notify.ts` – подписка клиентов и публикация событий;

– модели предметной области в `src/lib/models`.

В состав клиентской части входят страницы:

– `/` – описание назначения Lingua и переход к регистрации;

– `/login`, `/register` – вход и создание учётной записи;

– `/cabinet` – сводка;

– `/cabinet/words` – словарь;

– `/cabinet/lessons` – уроки;

– `/cabinet/progress` – прогресс и повторение.

3.2 Проектирование базы данных

В качестве хранилища выбрана документная СУБД MongoDB. Для учебной системы со словарями и уроками естественна схема без жёстких

соединений таблиц: каждая сущность хранится коллекцией документов, а связь с пользователем задаётся полем `userId` [3]. Состав коллекций приведён в таблице 3.1.

Таблица 3.1 – Коллекции базы данных Lingua

Коллекция	Назначение	Основные поля
users	учётные записи	name, email, passwordHash, nativeLanguage, targetLanguage
words	словарные записи	userId, term, translation, transcription, language, tags, mastery
lessons	уроки	userId, title, content, language, level
notices	уведомления	userId, title, message, kind, read

Поле `passwordHash` хранит результат bcrypt, исходный пароль в базе не сохраняется [4]. Для выборки записей пользователя создаётся индекс по `userId`. Поиск выполняется регулярным выражением без учёта регистра по полям термина, перевода, тегов, названия и текста урока.

3.3 Проектирование REST API

Взаимодействие клиента и сервера выполняется по REST. Идентификаторы передаются в пути, фильтры – в query, тела запросов – в JSON. Перечень основных маршрутов приведён в таблице 3.2.

Таблица 3.2 – Маршруты REST API

Метод и путь	Назначение
POST /api/auth/register	регистрация
POST /api/auth/login	вход
POST /api/auth/logout	выход
GET /api/auth/me	текущий пользователь
GET, POST /api/words	список и создание слов
GET, PATCH, DELETE /api/words/[id]	чтение, изменение, удаление слова
GET, POST /api/lessons	список и создание уроков
PATCH, DELETE /api/lessons/[id]	изменение и удаление урока
GET /api/progress	сводка прогресса
POST /api/translate	перевод
GET /api/notifications	журнал уведомлений
GET /api/notifications/stream	поток SSE

Все маршруты, кроме регистрации, входа и проверки работоспособности, требуют действительной сессии. Ответ 401 возвращается, если cookie отсутствует или JWT неверен.

3.4 Проектирование интерфейса

Главное окно кабинета состоит из шапки, навигации по разделам, рабочей области и журнала уведомлений. Шапка содержит знак приложения, имя пользователя и кнопку выхода. Навигация фиксирует активный раздел визуально. В разделе слов верхняя панель содержит поле поиска и списки сортировки, ниже расположена форма создания или редактирования, затем таблица. В разделе уроков записи отображаются карточками. На узкой ширине экрана сетки сворачиваются в один столбец, таблица получает горизонтальную прокрутку.

Цветовая схема строится на тёмно-синем основном цвете, тёплом бумажном фоне и акцентном терракотовом цвете для индикатора усвоения. Скругления блоков и тени используются умеренно, без декоративных иллюстраций, чтобы интерфейс оставался пригодным для учебной эксплуатации.

3.5 Алгоритмы функционирования

Алгоритмы представлены блок-схемами по ГОСТ 19.701–90 [6]. На схемах применяются терминаторы, процессы, решения и соединительные линии. Схемы выполнены в Visio. Рамка и основная надпись используются только на чертеже формата А1.

Запуск серверной части включает загрузку переменных окружения, подключение к MongoDB, инициализацию Route Handlers и прослушивание порта 3000. Последовательность показана на рисунке 3.1.

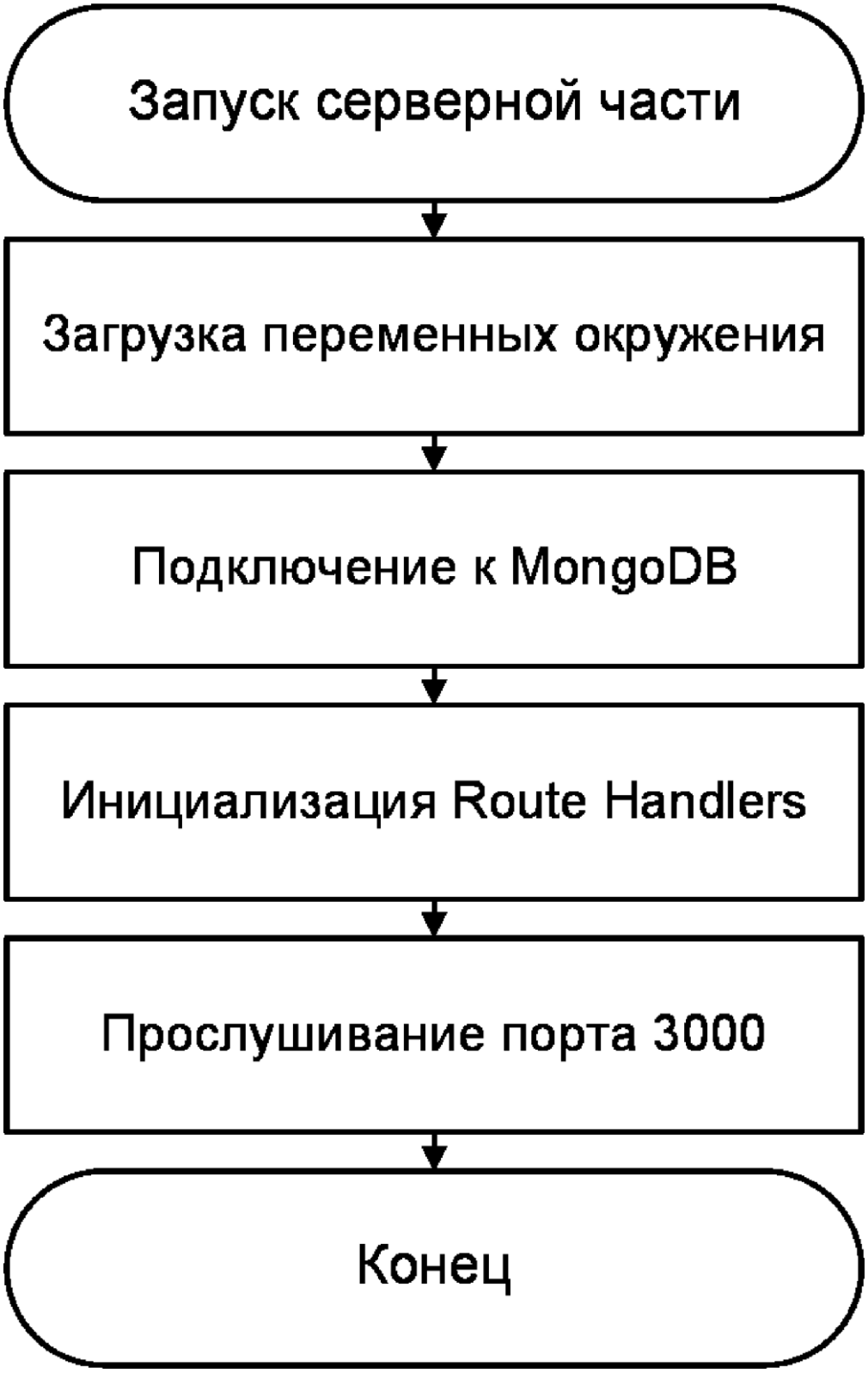


Рисунок 3.1 – Блок-схема запуска серверной части

Запуск клиентской части включает загрузку общего layout, маршрутов кабинета, запрос текущего пользователя и подписку на SSE. Последовательность показана на рисунке 3.2.



Рисунок 3.2 – Блок-схема запуска клиентской части

Регистрация начинается с ввода данных, затем выполняется валидация, проверка уникальности почты, хэширование пароля, сохранение документа и

выдача JWT. Ветви ошибок возвращают пользователя к исправлению ввода. Схема приведена на рисунке 3.3.

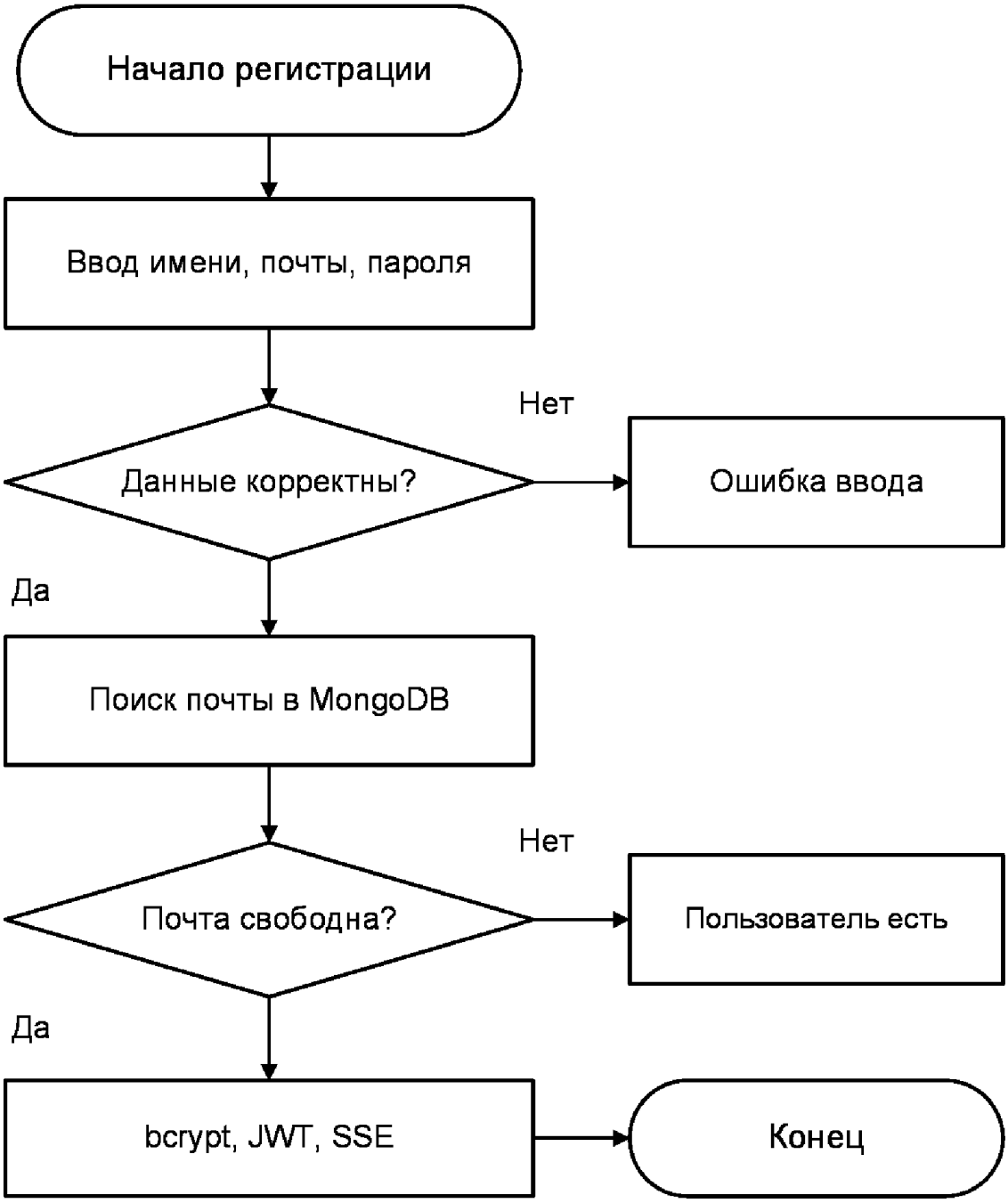


Рисунок 3.3 – Блок-схема регистрации пользователя

Создание словарной записи допускает автоперевод через MyMemory. После успешной валидации документ сохраняется, публикуется уведомление SSE и обновляется список. Схема приведена на рисунке 3.4.

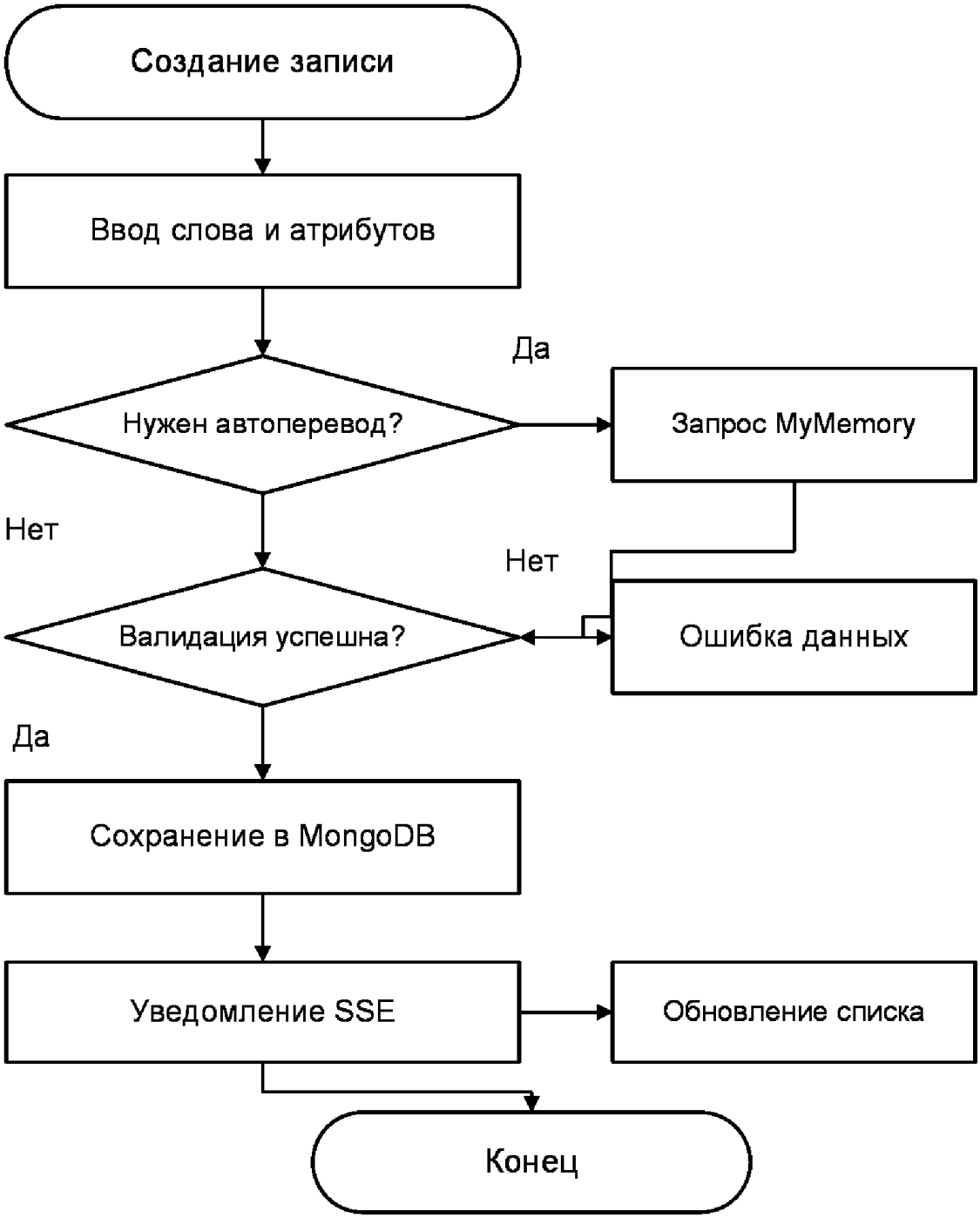


Рисунок 3.4 – Блок-схема создания словарной записи

Алгоритм перевода проверяет сессию и текст, выполняет HTTP-запрос к внешнему сервису и возвращает строку перевода либо код ошибки. Схема приведена на рисунке 3.5.

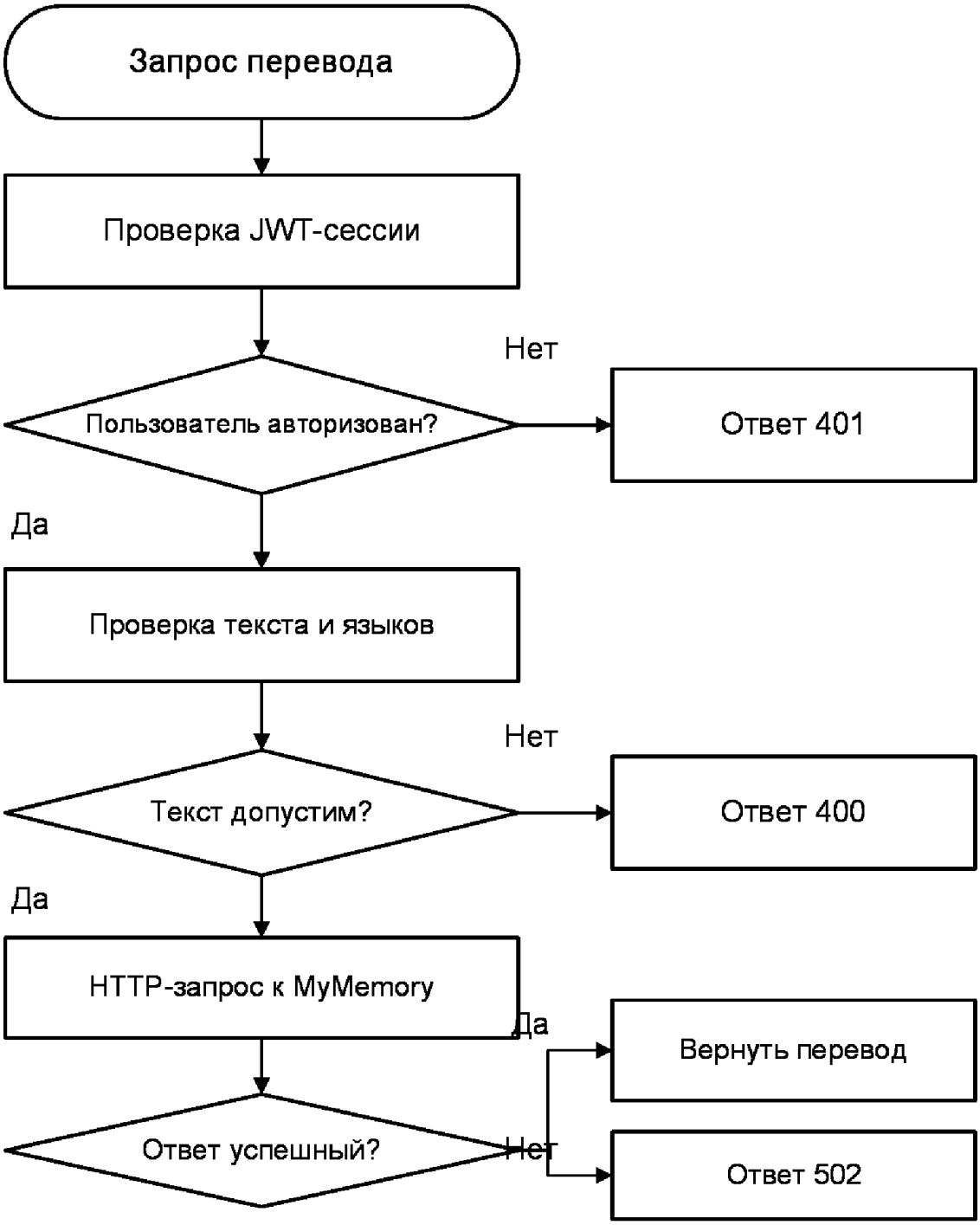


Рисунок 3.5 – Блок-схема получения перевода

Доставка уведомлений начинается с открытия потока SSE. При действительной сессии клиент регистрируется в памяти процесса. Новое событие записывается в коллекцию notices и отправляется в формате `data: JSON`. Схема приведена на рисунке 3.6.

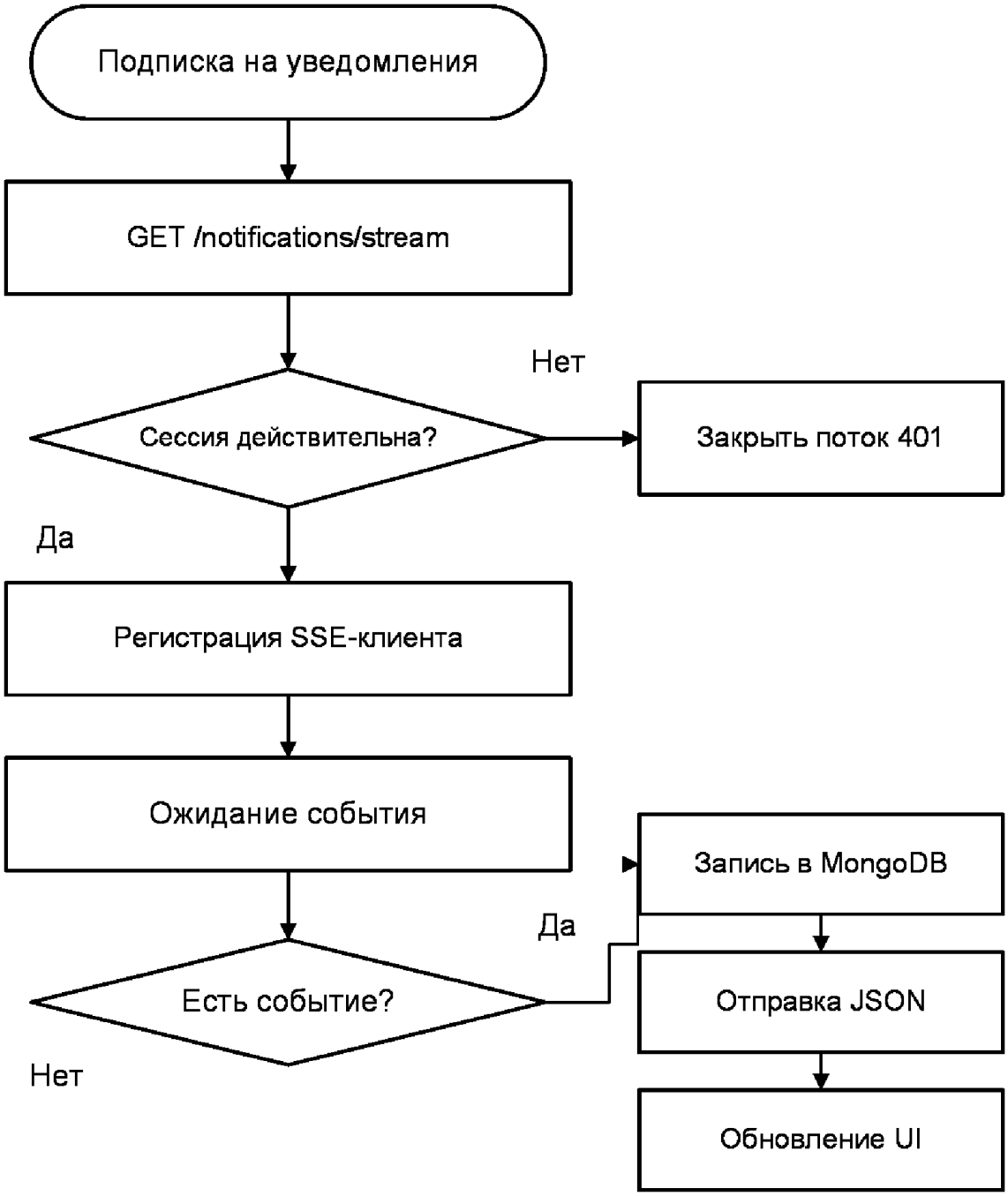


Рисунок 3.6 – Блок-схема доставки уведомлений SSE

Обобщённый алгоритм обработки запроса – проверка сессии, выбор Route Handler, обращение к MongoDB или MyMemory, обновление интерфейса и рассылка SSE – показан на рисунке 3.7. Развёрнутая схема с ветвями регистрации, CRUD, перевода и уведомлений вынесена на чертёж формата А1 с рамкой и основной надписью.

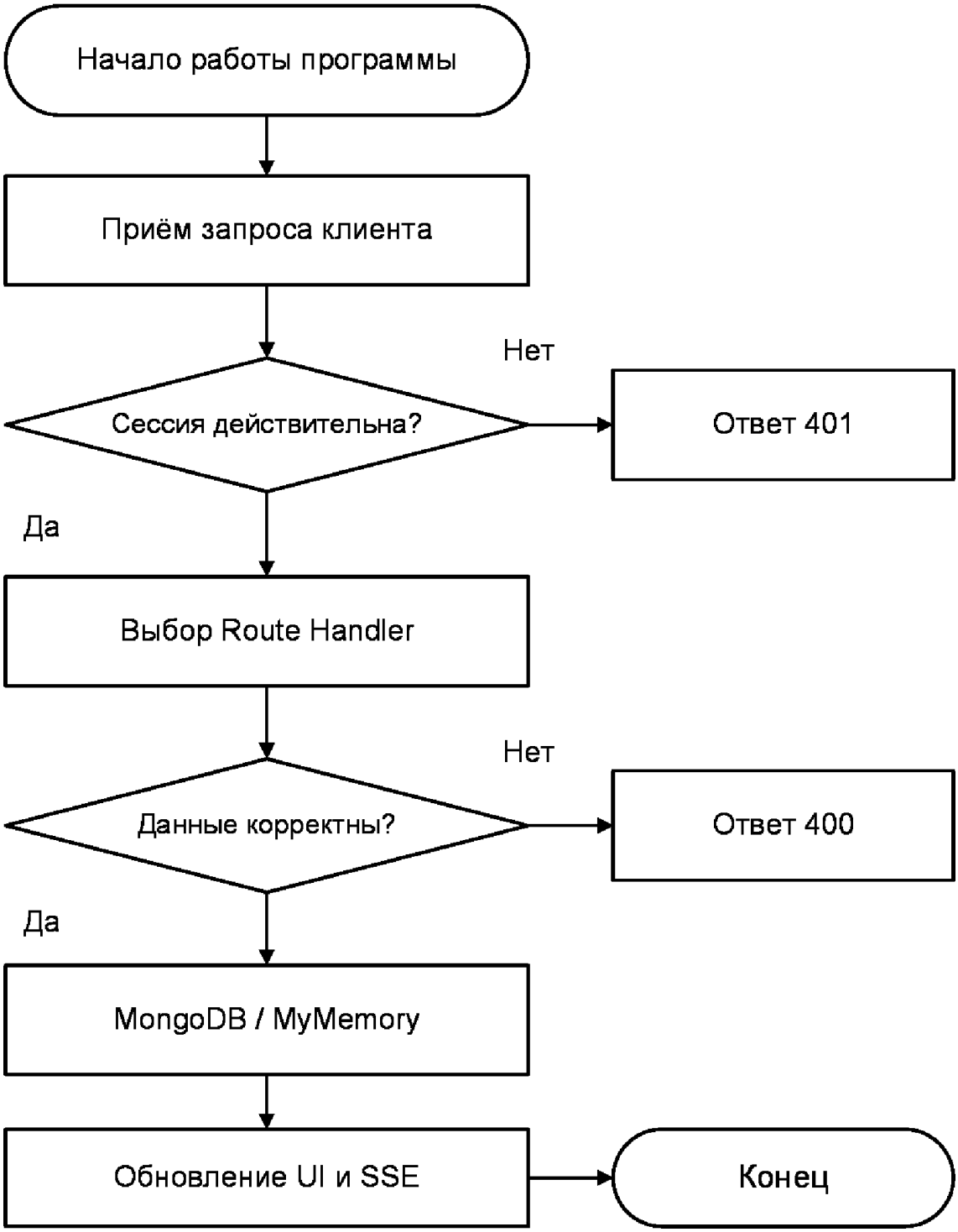


Рисунок 3.7 – Схема программы веб-приложения Lingua

4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА

4.1 Средства разработки

Конструирование выполнено в среде Windows с использованием Node.js, TypeScript и фреймворка Next.js 14. Серверная логика размещена в Route Handlers. Для доступа к MongoDB применена библиотека Mongoose. Пароли хэшируются bcryptjs, сессии подписываются JSON Web Token [2–4]. Контейнеризация выполнена Docker Compose: служба `mongo` поднимает СУБД, служба `app` собирает production-образ Next.js в режиме `standalone` [7].

Структура каталогов отражает разделение на страницы App Router, API и библиотечные модули. Листинг подключения к базе данных приведён ниже.

```
import mongoose from "mongoose";
import { seedIfEmpty } from "./seed";

const MONGODB_URI = process.env.MONGODB_URI ||
"mongodb://127.0.0.1:27017/lingua";

type Cache = {
  conn: typeof mongoose | null;
  promise: Promise<typeof mongoose> | null;
  seeded: boolean;
};

const globalWithCache = global as typeof globalThis & { _linguaMongo?: Cache
};

const cache: Cache = globalWithCache._linguaMongo || {
  conn: null,
  promise: null,
  seeded: false,
};

globalWithCache._linguaMongo = cache;

export async function connectDb() {
  if (cache.conn) {
    if (!cache.seeded) {
      await seedIfEmpty();
      cache.seeded = true;
    }
    return cache.conn;
  }

  if (!cache.promise) {
    cache.promise = (async () => {
      let lastError: unknown;
      for (let attempt = 1; attempt <= 20; attempt += 1) {
        try {
```

```
          return await mongoose.connect(MONGODB_URI, {
            serverSelectionTimeoutMS: 5000,
          });
        } catch (error) {
          lastError = error;
          await new Promise((resolve) => setTimeout(resolve, 2000));
        }
      }
      throw lastError;
    })();
  }

  cache.conn = await cache.promise;
  if (!cache.seeded) {
    await seedIfEmpty();
    cache.seeded = true;
  }
  return cache.conn;
}
```

4.2 Реализация аутентификации

Регистрация проверяет длину имени, формат почты и минимальную длину пароля. При конфликте почты возвращается код 409. Пароль преобразуется функцией `bcrypt.hash` с коэффициентом сложности 10. Идентификатор пользователя помещается в JWT и записывается в httpOnly-cookie `lingua_session` [8]. Вход сравнивает пароль с хэшем через `bcrypt.compare`. Извлечение текущего пользователя выполняется в `getCurrentUser` по cookie. Фрагмент модуля аутентификации приведён ниже.

```
import jwt from "jsonwebtoken";
import { cookies } from "next/headers";
import { NextResponse } from "next/server";
import { connectDb } from "./db";
import User from "./models/User";

const COOKIE = "lingua_session";

function secret() {
  return process.env.JWT_SECRET || "lingua-course-work-2026";
}

export function signToken(userId: string) {
  return jwt.sign({ sub: userId }, secret(), {
    expiresIn: (process.env.JWT_EXPIRES || "7d") as
jwt.SignOptions["expiresIn"],
  });
}

export function setSessionCookie(response: NextResponse, token: string) {
  response.cookies.set(COOKIE, token, {
    httpOnly: true,
    sameSite: "lax",
    path: "/",
    maxAge: 60 * 60 * 24 * 7,
  });
```

```
  return response;
}

export function clearSessionCookie(response: NextResponse) {
  response.cookies.set(COOKIE, "", { httpOnly: true, path: "/", maxAge: 0 });
  return response;
}

export async function getCurrentUser() {
  await connectDb();
  const token = cookies().get(COOKIE)?.value;
  if (!token) {
    return null;
  }
  try {
    const payload = jwt.verify(token, secret()) as { sub: string };
    const user = await User.findById(payload.sub);
    return user;
  } catch {
    return null;
  }
}

export function unauthorized() {
  return NextResponse.json({ error: "Требуется авторизация" }, { status: 401
});
}

export function publicUser(user: { _id: unknown; name: string; email: string;
nativeLanguage: string; targetLanguage: string }) {
  return {
    id: String(user._id),
    name: user.name,
    email: user.email,
    nativeLanguage: user.nativeLanguage,
    targetLanguage: user.targetLanguage,
  };
}
```

Обработчик регистрации размещён в Route Handler.

```
import bcrypt from "bcryptjs";
import { NextRequest, NextResponse } from "next/server";
import { connectDb } from "@/lib/db";
import User from "@/lib/models/User";
import { isEmail, requireString } from "@/lib/validate";
import { publicUser, setSessionCookie, signToken } from "@/lib/auth";
import { notify } from "@/lib/notify";

export const dynamic = "force-dynamic";


export async function POST(request: NextRequest) {
  await connectDb();
  const body = await request.json().catch(() => ({}));
  const errors = [
    requireString(body.name, "Имя", 2, 80),
    requireString(body.email, "Почта", 5, 120),
    requireString(body.password, "Пароль", 8, 72),
  ].filter(Boolean) as string[];
```

```
  if (body.email && !isEmail(String(body.email))) {
    errors.push("Почта: некорректный формат");
  }
  if (errors.length) {
    return NextResponse.json({ error: errors.join("; ") }, { status: 400 });
  }

  const email = String(body.email).trim().toLowerCase();
  const exists = await User.findOne({ email });
  if (exists) {
    return NextResponse.json({ error: "Пользователь с такой почтой уже
существует" }, { status: 409 });
  }

  const user = await User.create({
    name: String(body.name).trim(),
    email,
    passwordHash: await bcrypt.hash(String(body.password), 10),
    nativeLanguage: body.nativeLanguage || "ru",
    targetLanguage: body.targetLanguage || "en",
  });

  await notify(String(user._id), "Регистрация завершена", "Личный кабинет
готов к работе со словарём и уроками.", "success");
  const response = NextResponse.json({ user: publicUser(user) });
  return setSessionCookie(response, signToken(String(user._id)));
}
```

4.3 Реализация CRUD и поиска

Коллекция слов обслуживается маршрутами GET и POST `/api/words` и маршрутами GET, PATCH, DELETE `/api/words/[id]`. Поиск строится как логическое ИЛИ по полям `term`, `translation` и `tags`. Сортировка допускается только по белому списку полей, что исключает инъекцию произвольного ключа в `sort`. Аналогично устроены уроки. Пример обработчика списка и создания слов:

```
import { NextRequest, NextResponse } from "next/server";
import { getCurrentUser, unauthorized } from "@/lib/auth";
import Word from "@/lib/models/Word";
import { clampMastery, parseListQuery, requireString } from "@/lib/validate";
import { notify } from "@/lib/notify";

export const dynamic = "force-dynamic";


function serialize(doc: Record<string, unknown>) {
  return {
    id: String(doc._id),
    term: doc.term,
    translation: doc.translation,
    transcription: doc.transcription,
    language: doc.language,
    partOfSpeech: doc.partOfSpeech,
    example: doc.example,
```

```
    tags: doc.tags,
    mastery: doc.mastery,
    notes: doc.notes,
    createdAt: doc.createdAt,
    updatedAt: doc.updatedAt,
  };
}

export async function GET(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const { q, sort, order } = parseListQuery(new URL(request.url), "words");
  const filter: Record<string, unknown> = { userId: user._id };
  if (q) {
    filter.$or = [
      { term: { $regex: q, $options: "i" } },
      { translation: { $regex: q, $options: "i" } },
      { tags: { $regex: q, $options: "i" } },
    ];
  }
  const items = await Word.find(filter).sort({ [sort]: order }).lean();
  return NextResponse.json({ items: items.map(serialize) });
}

export async function POST(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const body = await request.json().catch(() => ({}));
  const errors = [
    requireString(body.term, "Слово", 1, 120),
    requireString(body.translation, "Перевод", 1, 240),
  ].filter(Boolean) as string[];
  if (errors.length) {
    return NextResponse.json({ error: errors.join("; ") }, { status: 400 });
  }

  const tags = Array.isArray(body.tags)
    ? body.tags.map((t: unknown) => String(t).trim()).filter(Boolean)
    : String(body.tags || "")
        .split(",")
        .map((t) => t.trim())
        .filter(Boolean);

  const created = await Word.create({
    userId: user._id,
    term: String(body.term).trim(),
    translation: String(body.translation).trim(),
    transcription: String(body.transcription || "").trim(),
    language: String(body.language || user.targetLanguage || "en").trim(),
    partOfSpeech: String(body.partOfSpeech || "").trim(),
    example: String(body.example || "").trim(),
    notes: String(body.notes || "").trim(),
    tags,
    mastery: clampMastery(body.mastery),
  });

  await notify(String(user._id), "Слово добавлено", `«${created.term}»
сохранено в словаре.`, "success");
```

```
  return NextResponse.json({ item: serialize(created.toObject()) }, { status:
201 });
}
```

Схема документа Word задаёт ограничения длины и диапазон `mastery`.

```
import mongoose, { Schema } from "mongoose";

const WordSchema = new Schema(
  {
    userId: { type: Schema.Types.ObjectId, ref: "User", required: true,
index: true },
    term: { type: String, required: true, trim: true, maxlength: 120 },
    translation: { type: String, required: true, trim: true, maxlength: 240
},
    transcription: { type: String, default: "", trim: true, maxlength: 80 },
    language: { type: String, default: "en", trim: true, maxlength: 8 },
    partOfSpeech: { type: String, default: "", trim: true, maxlength: 40 },
    example: { type: String, default: "", trim: true, maxlength: 400 },
    tags: { type: [String], default: [] },
    mastery: { type: Number, default: 0, min: 0, max: 5 },
    notes: { type: String, default: "", maxlength: 400 },
  },
  { timestamps: true }
);

WordSchema.index({ userId: 1, term: 1 });

export default mongoose.models.Word || mongoose.model("Word", WordSchema);
```

4.4 Интеграция со сторонним сервисом перевода

Перевод выполняется обработчиком POST `/api/translate`. После проверки сессии и текста формируется URL сервиса MyMemory с параметрами `q` и `langpair`. Ответ внешнего API разбирается, из поля `responseData.translatedText` извлекается строка. Таймауты и неуспешные коды превращаются в ответ 502 с сообщением на русском языке. Клиент словаря по кнопке «Перевести» подставляет полученное значение в поле перевода, пользователь может его отредактировать перед сохранением.

```
import { NextRequest, NextResponse } from "next/server";
import { getCurrentUser, unauthorized } from "@/lib/auth";
import { requireString } from "@/lib/validate";

export const dynamic = "force-dynamic";


export async function POST(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const body = await request.json().catch(() => ({}));
  const err = requireString(body.text, "Текст", 1, 500);
```

```
  if (err) {
    return NextResponse.json({ error: err }, { status: 400 });
  }
  const from = String(body.from || "en").trim();
  const to = String(body.to || "ru").trim();
  const endpoint = process.env.TRANSLATE_ENDPOINT ||
"https://api.mymemory.translated.net/get";
  const url =
`${endpoint}?q=${encodeURIComponent(String(body.text).trim())}&langpair=${enc
odeURIComponent(`${from}|${to}`)}`;

  try {
    const res = await fetch(url, { cache: "no-store" });
    if (!res.ok) {
      return NextResponse.json({ error: "Сервис перевода временно недоступен"
}, { status: 502 });
    }
    const data = await res.json();
    const translation = data?.responseData?.translatedText;
    if (!translation) {
      return NextResponse.json({ error: "Не удалось получить перевод" }, {
status: 502 });
    }
    return NextResponse.json({ translation, provider: "MyMemory" });
  } catch {
    return NextResponse.json({ error: "Ошибка обращения к сервису перевода"
}, { status: 502 });
  }
}
```

4.5 Реализация уведомлений SSE

Модуль `sse.ts` хранит множество функций отправки, индексированное идентификатором пользователя. Функция `notify` сначала создаёт документ Notice, затем сериализует его в кадр SSE. Маршрут `/api/notifications/stream` открывает `ReadableStream` с заголовком `text/event-stream` [9]. На клиенте объект `EventSource` добавляет событие в журнал и во всплывающее сообщение. Таким образом удовлетворяется требование задания об уведомлениях через Route Handler.

```
type Client = {
  userId: string;
  send: (payload: string) => void;
};

const clients = new Map<string, Set<Client>>();

export function subscribe(userId: string, send: (payload: string) => void) {
  const client: Client = { userId, send };
  const set = clients.get(userId) || new Set<Client>();
  set.add(client);
  clients.set(userId, set);
  return () => {
    set.delete(client);
    if (set.size === 0) {
      clients.delete(userId);
```

```
    }
  };
}

export function pushEvent(userId: string, data: unknown) {
  const payload = `data: ${JSON.stringify(data)}\n\n`;
  const set = clients.get(userId);
  if (!set) {
    return;
  }
  for (const client of set) {
    try {
      client.send(payload);
    } catch {
      set.delete(client);
    }
  }
}
import { getCurrentUser } from "@/lib/auth";
import { subscribe } from "@/lib/sse";

export const dynamic = "force-dynamic";

export async function GET() {
  const user = await getCurrentUser();
  if (!user) {
    return new Response("Требуется авторизация", { status: 401 });
  }

  const encoder = new TextEncoder();
  let cleanup: () => void = () => undefined;

  const stream = new ReadableStream({
    start(controller) {
      const send = (payload: string) =>
controller.enqueue(encoder.encode(payload));
      cleanup = subscribe(String(user._id), send);
      send(`data: ${JSON.stringify({ type: "hello" })}\n\n`);
    },
    cancel() {
      cleanup();
    },
  });

  return new Response(stream, {
    headers: {
      "Content-Type": "text/event-stream; charset=utf-8",
      "Cache-Control": "no-cache, no-transform",
      Connection: "keep-alive",
    },
  });
}
```

4.6 Адаптивная верстка

Стили собраны в `globals.css` без внешних UI-библиотек, чтобы оформление оставалось предсказуемым в контейнере [10]. Сетка героя, карточек и статистики переходит к одному столбцу при ширине менее 900 px. Шапка на узком экране выстраивается вертикально: знак приложения, имя пользователя и кнопка выхода не накладываются друг на друга. Кнопки и поля ввода имеют высоту не менее 40 px, чтобы их можно было нажать пальцем.

Таблица словаря при узкой ширине получает горизонтальную прокрутку, заголовки столбцов сохраняются. Карточки уроков занимают всю ширину колонки, длинный текст урока переносится по словам. Журнал уведомлений на мобильной ширине располагается под рабочей областью, а не сбоку, поэтому основные действия остаются в зоне первого экрана.

5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

5.1 Виды тестирования

Проверка работоспособности включала функциональное тестирование REST API и пользовательского интерфейса, негативные сценарии валидации, проверку изоляции данных между учётными записями и визуальную оценку адаптивности. Тесты выполнялись на локальном контуре Docker Compose: MongoDB в отдельном контейнере, приложение на порту 3000. Для запросов API использовались браузер и утилита curl.

5.2 Тест-кейсы

Основные тест-кейсы сведены в таблицу 5.1. Ожидаемый результат формулировался до выполнения. Фактический результат совпал с ожидаемым во всех приведённых случаях.

Таблица 5.1 – Результаты функционального тестирования

Идентификатор	Действие	Ожидаемый результат	Факт
ТК-1	Регистрация с паролем из 4 символов	Код 400, сообщение о длине пароля	совпало
ТК-2	Регистрация с некорректной почтой	Код 400	совпало
ТК-3	Повторная регистрация той же почты	Код 409	совпало
ТК-4	Вход demo@lingua.local / Demo12345	Переход в кабинет, cookie сессии	совпало
ТК-5	Вход с неверным паролем	Код 401	совпало
ТК-6	GET /api/words без cookie	Код 401	совпало
ТК-7	Создание слова без перевода	Код 400	совпало
ТК-8	Создание слова apple / яблоко	Запись в таблице, уведомление SSE	совпало
ТК-9	Поиск по подстроке «learn»	Отфильтрованный список	совпало

Идентификатор	Действие	Ожидаемый результат	Факт
ТК-10	Сортировка слов по mastery	Порядок изменяется	совпало
ТК-11	Изменение mastery на 5	Значение сохраняется	совпало
ТК-12	Удаление слова	Запись исчезает, уведомление	совпало
ТК-13	Создание урока короче 10 символов	Код 400	совпало
ТК-14	Создание урока уровня B1	Карточка в списке	совпало
ТК-15	Перевод слова resilient	Заполнено поле перевода	совпало
ТК-16	Повторение слова с верным ответом	mastery увеличивается	совпало
ТК-17	Открытие кабинета на ширине 390 px	Блоки в один столбец	совпало

Дополнительно проверена изоляция: слово, созданное первой учётной записью, не возвращается в GET `/api/words` второй учётной записи. Попытка PATCH чужого идентификатора даёт код 404, так как фильтр включает `userId`.

5.3 Анализ результатов

Тестирование подтвердило выполнение требований задания: REST API функционирует, нереляционная база используется для всех сущностей, реализованы вход и регистрация, CRUD, поиск и сортировка, валидация и bcrypt, адаптивная вёрстка, интеграция с MyMemory и уведомления SSE. Ограничением учебной реализации является хранение подписчиков SSE в памяти процесса: при нескольких экземплярах приложения потребуется внешняя шина. Для курсового контура с одним контейнером `app` ограничение приемлемо.

Демонстрационная учётная запись заполняется при первом подключении к пустой базе, что ускоряет проверку словаря и уроков без ручного ввода начальных данных.

6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ

6.1 Установка и запуск

Для запуска необходимы Docker и Docker Compose. Исходные файлы размещаются в каталоге `lang-app`. В корне каталога находятся `Dockerfile` и `docker-compose.yml`. Запуск выполняется командой

```
docker compose up --build
```

После сборки образа приложение доступно по адресу http://localhost:3000. Служба MongoDB не публикуется наружу и доступна приложению по имени хоста `mongo`. Переменные `MONGODB_URI`, `JWT_SECRET` и `TRANSLATE_ENDPOINT` задаются в `docker-compose.yml`. Стартовая страница приведена на рисунке 6.1.

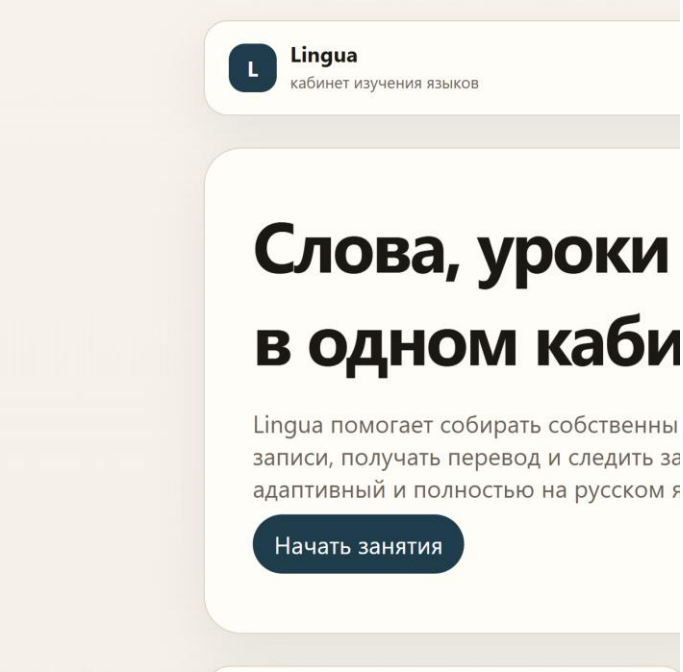


Рисунок 6.1 – Стартовая страница приложения Lingua

При первом старте, если коллекция пользователей пуста, создаётся демонстрационная учётная запись:

– почта: demo@lingua.local;

– пароль: Demo12345.

Остановка выполняется сочетанием Ctrl+C в том же терминале либо командой `docker compose down`. Данные MongoDB сохраняются в томе `lingua_mongo`.

6.2 Руководство пользователя

6.2.1 Регистрация и вход. На стартовой странице выбирается «Регистрация». Форма регистрации показана на рисунке 6.2. Вводятся имя, почта, пароль и изучаемый язык. После успешного сохранения открывается кабинет. Для возврата используется «Вход». Форма входа показана на рисунке 6.3.

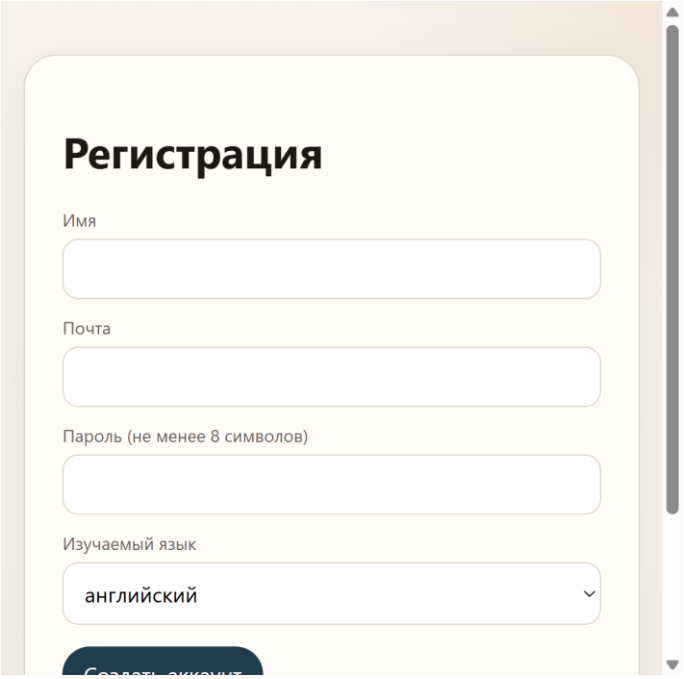


Рисунок 6.2 – Форма регистрации



Рисунок 6.3 – Форма входа

6.2.2 Работа со словарём. В разделе «Слова» заполняются термин и перевод. При необходимости нажимается «Перевести»: запрос уходит на сервер и во внешний сервис MyMemory, результат подставляется в поле перевода. Дополнительно задаются транскрипция, часть речи, пример, теги и уровень усвоения. Кнопка «Добавить» создаёт запись. Поиск фильтрует таблицу, списки справа меняют поле и направление сортировки. Команды «Изменить» и «Удалить» обслуживают выбранную строку.

6.2.3 Работа с уроками. В разделе «Уроки» задаются название, уровень A1–C2, язык и текст. После создания урок отображается карточкой. Редактирование возвращает значения в форму, удаление требует подтверждения.

6.2.4 Прогресс и повторение. Раздел «Прогресс» показывает число слов, число освоенных карточек, число уроков и распределение по языкам. Форма повторения предлагает термин; пользователь вводит перевод. При совпадении уровень усвоения увеличивается, при ошибке уменьшается.

6.2.5 Уведомления. Журнал в нижней части кабинета и всплывающие сообщения отображают создание, изменение и удаление записей. Поток SSE открывается автоматически при входе в кабинет.

6.2.6 Завершение работы. Кнопка «Выйти» очищает cookie сессии и возвращает на стартовую страницу.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработано веб-приложение Lingua для изучения иностранных языков. Проанализированы предметная область и аналоги Duolingo, Lingualeo, Quizlet и Anki, сформулированы функциональные и нефункциональные требования, спроектированы архитектура, документная база данных, REST API, интерфейс кабинета и алгоритмы основных сценариев.

Реализованы регистрация и вход с проверкой данных и хэшированием паролей bcrypt, CRUD-операции для слов и уроков, поиск и сортировка, личный кабинет с разделами словаря, уроков и прогресса, интеграция со сторонним сервисом перевода MyMemory, уведомления по SSE и адаптивная вёрстка. Приложение упаковано в Docker Compose и запускается одной командой.

Тестирование подтвердило работоспособность основных сценариев, включая негативные случаи валидации и изоляцию данных пользователей. Графический материал содержит схему программы на листе формата А1. Дальнейшее развитие может включать интервальное повторение по расписанию, импорт словарей и горизонтальное масштабирование канала уведомлений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Сысоев, П. В. Информационные и коммуникационные технологии в обучении иностранному языку : теория и практика / П. В. Сысоев. – М. : Глосса-Пресс, 2012. – 238 с.

[2] Next.js Documentation [Электронный ресурс]. – Режим доступа : https://nextjs.org/docs.

[3] MongoDB Manual [Электронный ресурс]. – Режим доступа : https://www.mongodb.com/docs/manual/.

[4] bcryptjs [Электронный ресурс]. – Режим доступа : https://www.npmjs.com/package/bcryptjs.

[5] MyMemory translated.net API [Электронный ресурс]. – Режим доступа : https://mymemory.translated.net/doc/spec.php.

[6] ГОСТ 19.701–90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. – Введ. 01.01.92. – М. : Изд-во стандартов, 1991. – 21 с.

[7] Docker Compose [Электронный ресурс]. – Режим доступа : https://docs.docker.com/compose/.

[8] Jones, M. JSON Web Token (JWT) : RFC 7519 / M. Jones, J. Bradley, N. Sakimura. – IETF, 2015.

[9] HTML Living Standard. Server-sent events [Электронный ресурс]. – Режим доступа : https://html.spec.whatwg.org/multipage/server-sent-events.html.

[10] Фримен, Э. Изучаем программирование на JavaScript / Э. Фримен, Э. Робсон. – СПб. : Питер, 2015. – 640 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинги программного средства

В приложении А приведены дополнительные листинги конфигурации и моделей данных. Основные фрагменты серверной логики рассмотрены в разделе 4.

Листинг А.1 – Файл docker-compose.yml

```
services:
  mongo:
    image: mongo:7
    restart: unless-stopped
    volumes:
      - lingua_mongo:/data/db
  app:
    build: .
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      MONGODB_URI: mongodb://mongo:27017/lingua
      JWT_SECRET: lingua-course-work-2026
      JWT_EXPIRES: 7d
      TRANSLATE_ENDPOINT: https://api.mymemory.translated.net/get
    depends_on:
      - mongo

volumes:
  lingua_mongo:
```

Листинг А.2 – Модель пользователя

```
import mongoose, { Schema } from "mongoose";

const UserSchema = new Schema(
  {
    name: { type: String, required: true, trim: true, maxlength: 80 },
    email: { type: String, required: true, unique: true, lowercase: true,
trim: true },
    passwordHash: { type: String, required: true },
    nativeLanguage: { type: String, default: "ru" },
    targetLanguage: { type: String, default: "en" },
  },
  { timestamps: true }
);

export default mongoose.models.User || mongoose.model("User", UserSchema);
```

Листинг А.3 – Модель урока

```
import mongoose, { Schema } from "mongoose";
```

```
const LessonSchema = new Schema(
  {
    userId: { type: Schema.Types.ObjectId, ref: "User", required: true,
index: true },
    title: { type: String, required: true, trim: true, maxlength: 160 },
    content: { type: String, required: true, trim: true, maxlength: 8000 },
    language: { type: String, default: "en", trim: true, maxlength: 8 },
    level: { type: String, default: "A1", enum: ["A1", "A2", "B1", "B2",
"C1", "C2"] },
  },
  { timestamps: true }
);

export default mongoose.models.Lesson || mongoose.model("Lesson",
LessonSchema);
```

Листинг А.4 – Проверка входных данных

```
export function isEmail(value: string) {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
}

export function requireString(value: unknown, field: string, min = 1, max =
500) {
  if (typeof value !== "string") {
    return `${field}: значение должно быть строкой`;
  }
  const trimmed = value.trim();
  if (trimmed.length < min) {
    return `${field}: минимум ${min} символ(ов)`;
  }
  if (trimmed.length > max) {
    return `${field}: максимум ${max} символов`;
  }
  return null;
}

export function clampMastery(value: unknown) {
  const n = Number(value);
  if (!Number.isFinite(n)) {
    return 0;
  }
  return Math.max(0, Math.min(5, Math.round(n)));
}

const SORT_WORDS = new Set(["term", "translation", "createdAt", "updatedAt",
"mastery", "language"]);
const SORT_LESSONS = new Set(["title", "level", "language", "createdAt",
"updatedAt"]);

export function parseListQuery(url: URL, kind: "words" | "lessons") {
  const q = (url.searchParams.get("q") || "").trim();
  const sortRaw = url.searchParams.get("sort") || "createdAt";
  const allowed = kind === "words" ? SORT_WORDS : SORT_LESSONS;
  const sort = allowed.has(sortRaw) ? sortRaw : "createdAt";
  const order = url.searchParams.get("order") === "asc" ? 1 : -1;
  return { q, sort, order };
}
```